

Regression: Modeling Relationships

MKT 566 · Fall 2026

Davide Proserpio

USC Marshall School of Business

Before We Start

Housekeeping

- The **RateBeer report** ([ratebeer-report.html](#) + [ai-log.md](#)) is due **Sunday, Sept. 20, 11:59 pm** on Brightspace
- **Homework 2** is released today and due **Thursday, Oct. 8**. It covers today (OLS) and Tuesday (logit), on a box-office dataset
- **Group project**: proposal presentations are on **Oct. 13 and 15** (10–15 minutes per group: the problem you will study and how)
- Tuesday: **logit**, then the **Airbnb exercise** on your laptop

From charts to models

Last week you *looked* at how two variables move together. Today you put a **number** on it, and learn to tell when that number can be trusted.

Today you learn to

1. Say what a regression **is** (a line, chosen a specific way)
2. **Read** R's output: coefficient, standard error, p-value, R^2
3. **Interpret** a coefficient in business units, with and without logs
4. Handle **categorical** predictors (city, room type)

What stays the same

- The AI writes the code. You choose the model, and you read the result
- Two habits: **predict** before you run, **explain back** one line after
- Same data as before: the 200 ad campaigns from week 3, then 50,000 Airbnb listings

Regression is the workhorse of marketing analytics. Homework 2, the Airbnb exercise, and most group projects are built on what we cover today.

R script for today

In the [code/](#) folder on the course website:

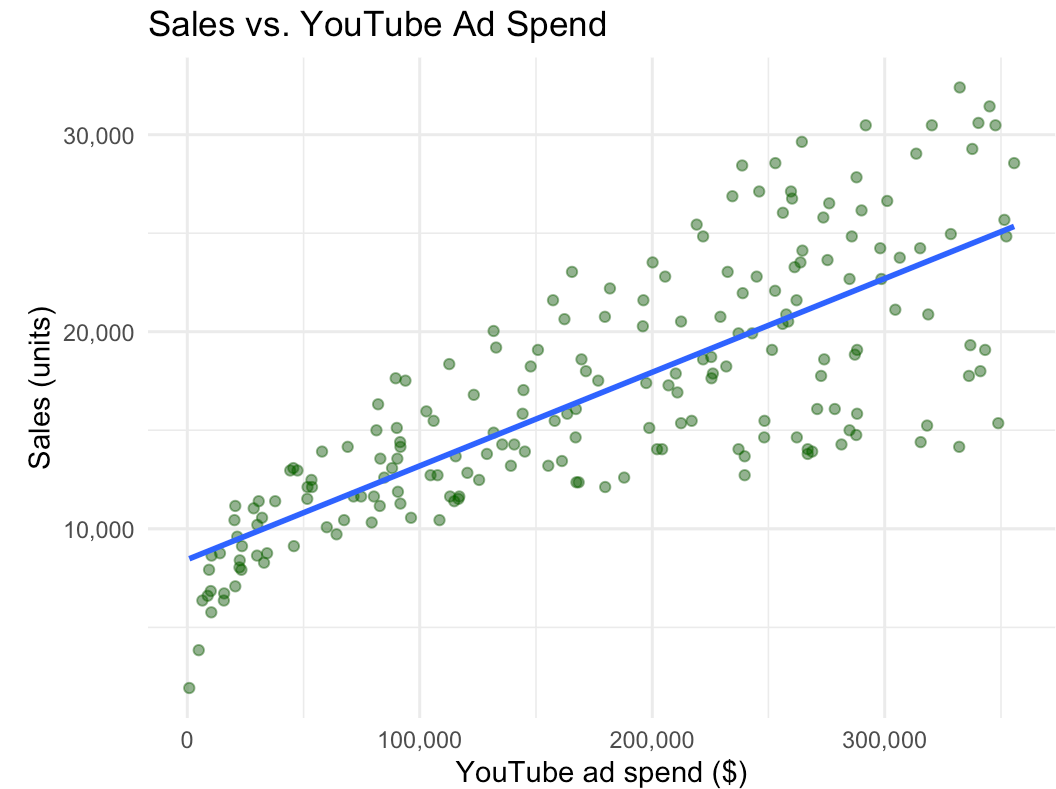
1. [w4-1-ols-class.R](#): every chart and every regression in today's deck, with a comment above each one
2. [data/airbnb.csv](#): the Airbnb dataset used in the second half (and in the Airbnb exercise on Tuesday, Sept. 22)

Download everything as one zip: [w4-code.zip](#), open the folder in VS Code, and run along.

From a Line to a Model

The line we drew last week

```
1 ggplot(marketing, aes(x = youtube, y = sales))
2   geom_point(alpha = 0.5, color = "darkgreen")
3   geom_smooth(method = "lm", se = FALSE) +
4   scale_x_continuous(labels = comma) +
5   scale_y_continuous(labels = comma) +
6   labs(title = "Sales vs. YouTube Ad Spend",
7         x = "YouTube ad spend ($)",
8         y = "Sales (units)") +
9   theme_minimal()
```



We asked R for a straight line through the points. What line is it? What could a manager do with it?

A regression is the formula for that line

$$\text{sales} = \beta_0 + \beta_1 \times \text{youtube}$$

- A regression **predicts** one variable, Y , from one or more others, X
- Y is the **outcome** (or dependent variable). X is the **predictor** (or independent variable)
- Examples:
 - Sales (Y) as a function of ad spend (X)
 - Revenue (Y) as a function of review ratings (X)
 - Airbnb price (Y) as a function of the number of bedrooms (X)
 - Income (Y) as a function of years of education (X)

In general $Y = F(X)$: Y depends on X somehow. A **linear** regression assumes the simplest F : a straight line. Machine learning (week 10) is mostly about learning fancier F 's.

The setup

For each row i of the data (each campaign):

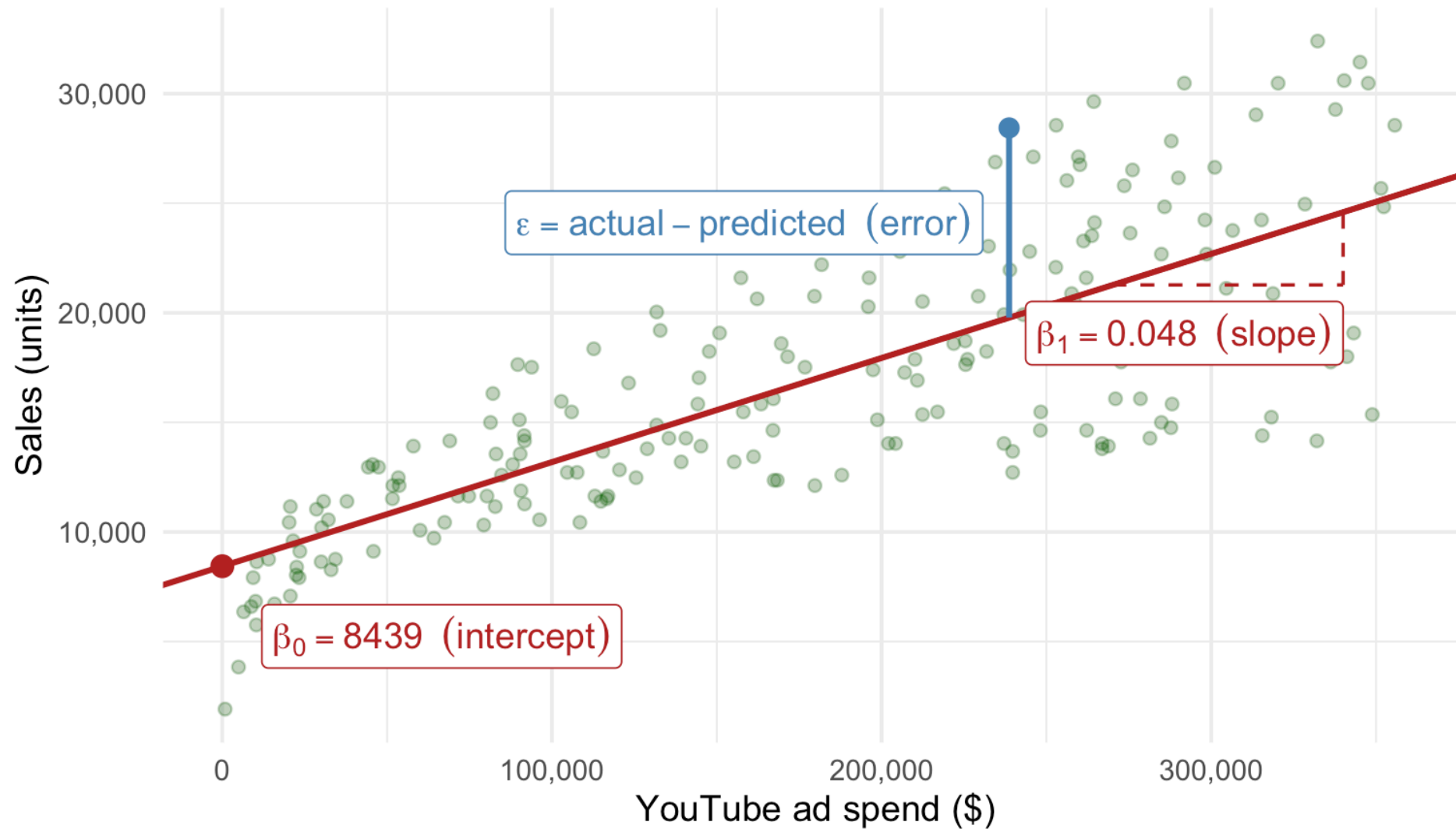
$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

- x_i : the predictor (YouTube spend of campaign i)
- y_i : the outcome (sales of campaign i)
- β_0 : the **intercept**, predicted Y when $X = 0$
- β_1 : the **slope**, how much Y changes when X goes up by one unit
- ϵ_i : the **error**, the part of y_i the line does not explain

x = youtube (\$)	y = sales (units)
276,120	26,520
53,400	12,480
20,640	11,160
181,800	22,200
216,960	15,480

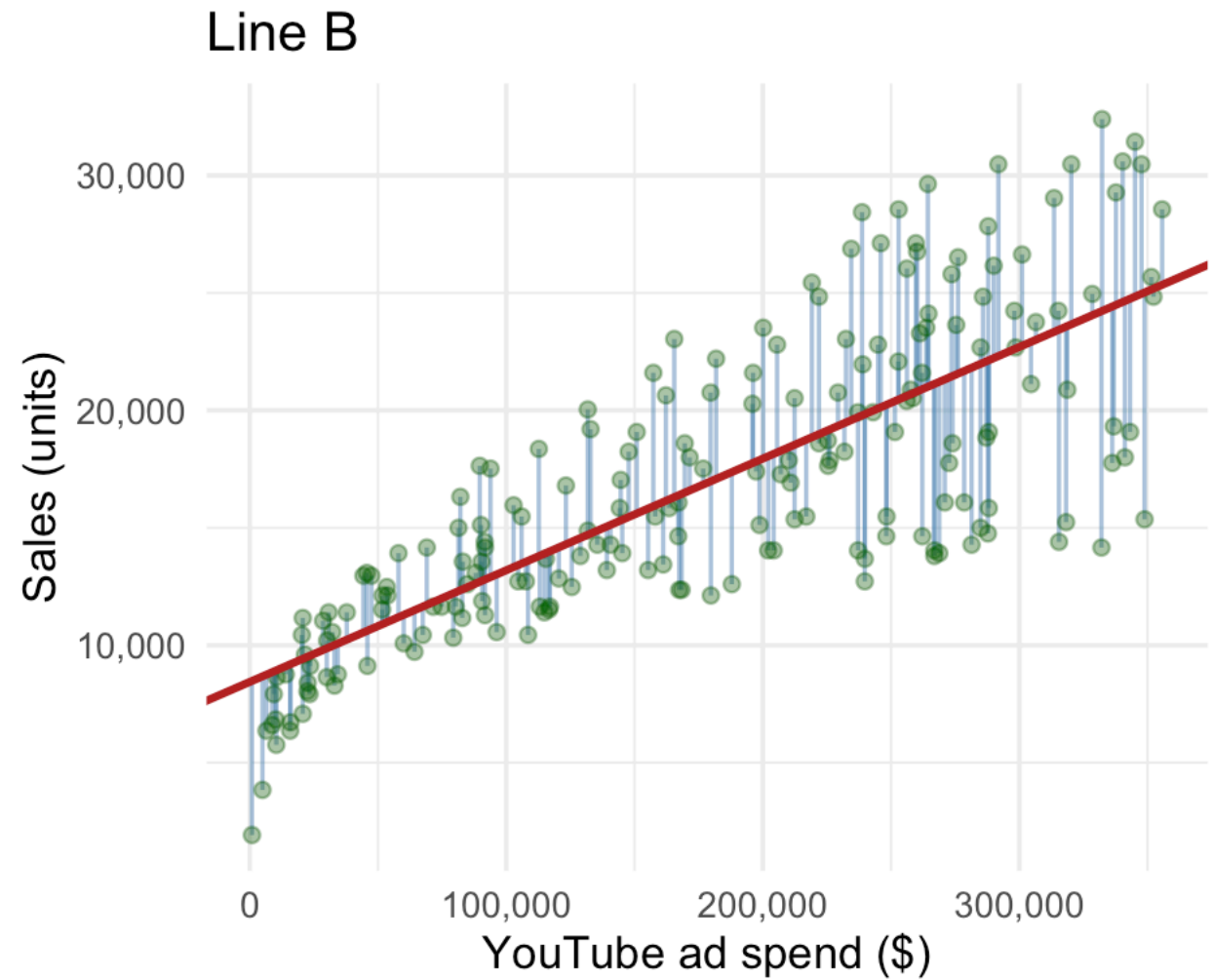
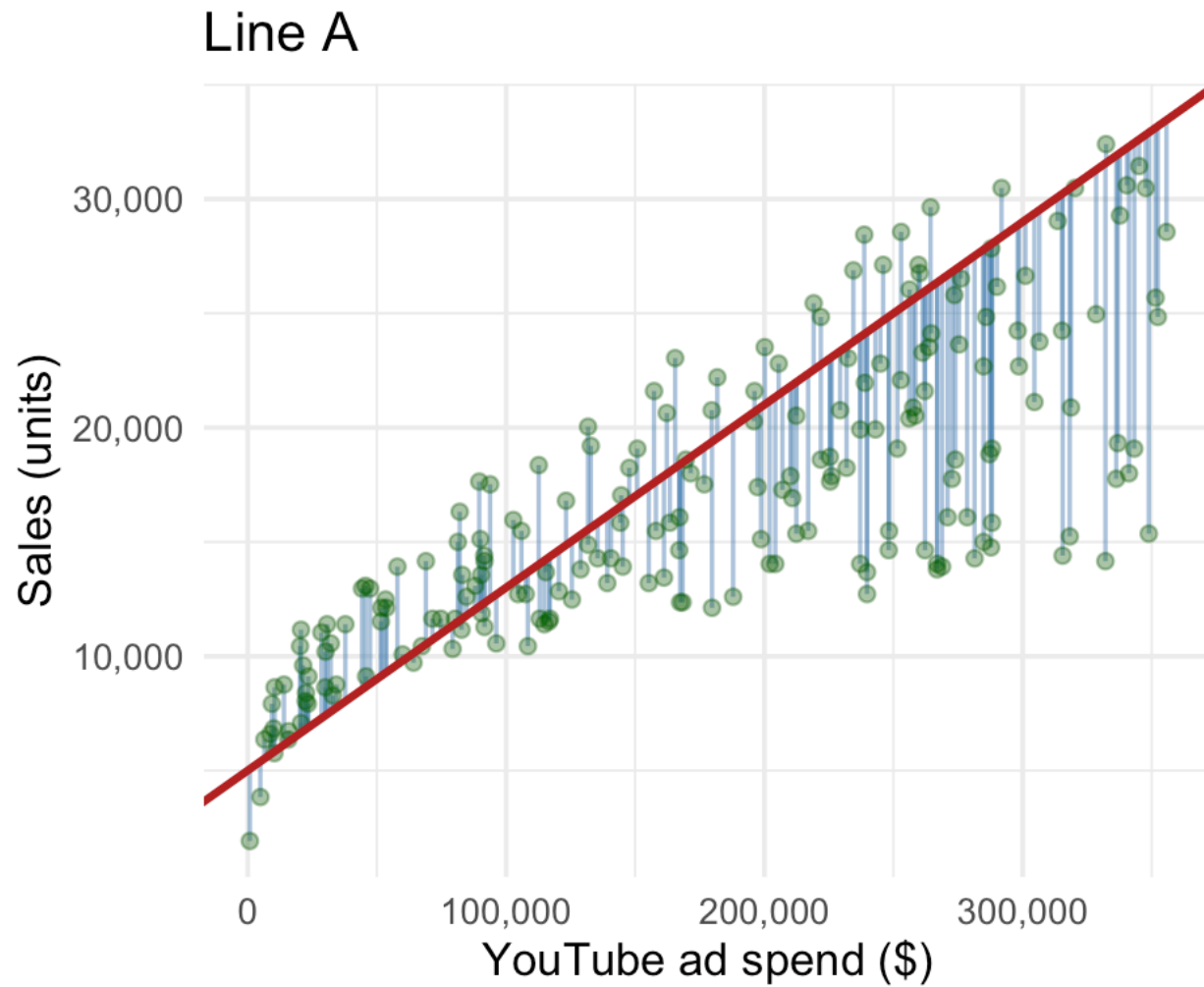
Rows $i = 1, \dots, 200$

The three pieces, on the chart



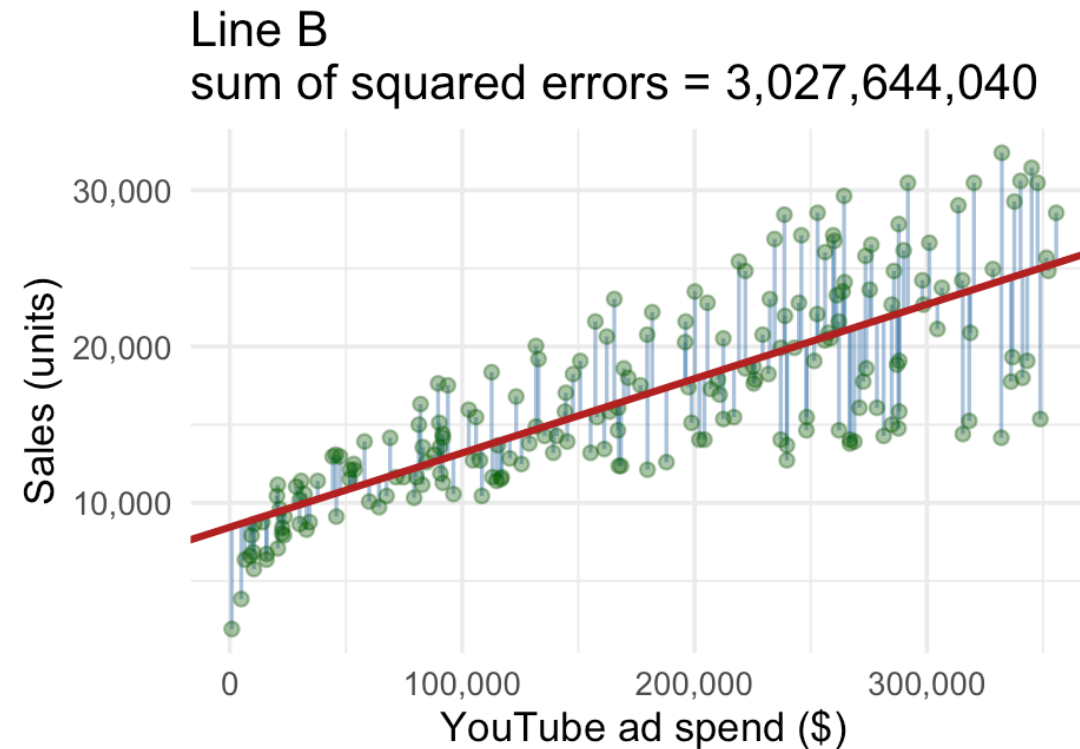
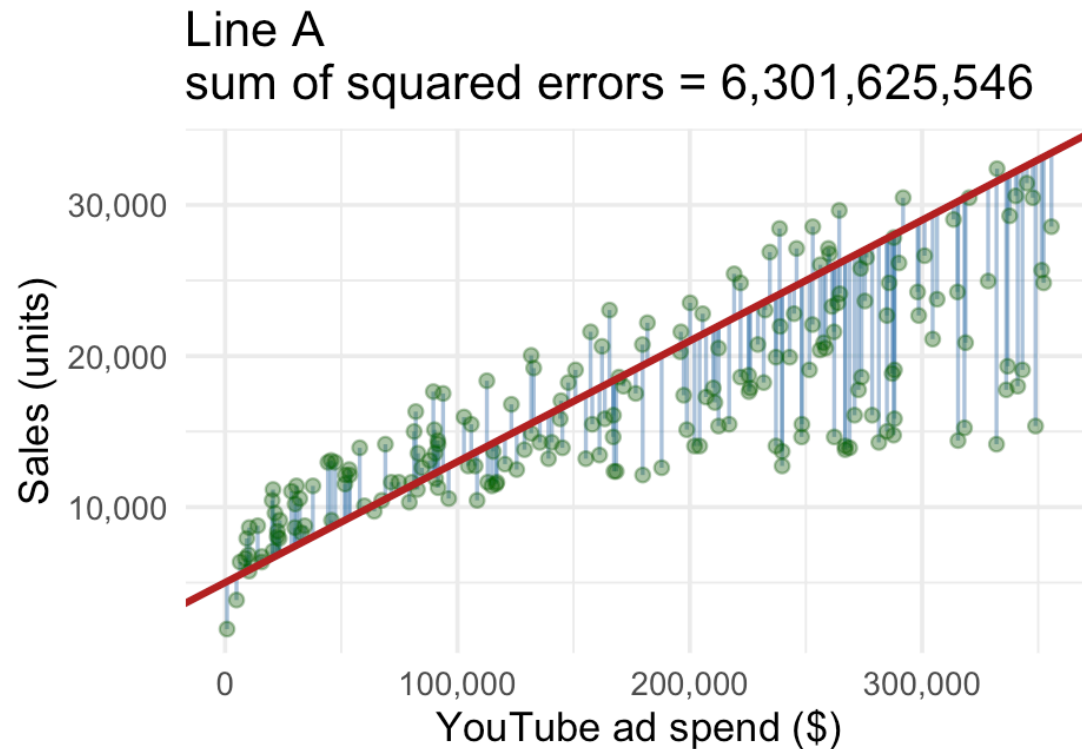
Every point has its own error: the vertical distance to the line. The intercept and the slope are the same for everyone.

Which line?



Both lines run through the points. The blue segments are the errors. Which line is better, and how would you *decide*?

Which line: reading it



- Square each error (so misses above and below the line both count), add them up. Smaller is better
- **Ordinary least squares (OLS)** picks the intercept and slope with the **smallest** sum of squared errors. Line B is that line, and it is what `geom_smooth(method = "lm")` drew last week
- “Least squares” is the whole idea. Everything else today is about reading what comes out

The sum of squared errors, written out

For any intercept β_0 and slope β_1 , the line predicts $\hat{y}_i = \beta_0 + \beta_1 x_i$ and misses by $\epsilon_i = y_i - \hat{y}_i$:

$$\text{SSE}(\beta_0, \beta_1) = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

- Every choice of (β_0, β_1) gives a different line, hence a different SSE. Line A and Line B are two such choices
- OLS picks the pair that makes this sum as small as possible:

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \text{SSE}(\beta_0, \beta_1)$$

“Least squares” means exactly this: square each error, add them up, and pick the line that makes the total smallest.

x_i	y_i	$\hat{y}_i$	ϵ_i	ϵ_i^2
276,120	26,520	21,565	4,955	24,552,724
53,400	12,480	10,978	1,502	2,257,299
20,640	11,160	9,420	1,740	3,026,666
181,800	22,200	17,081	5,119	26,201,361
⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	3,027,644,040

Line B (the OLS line), first 4 of 200 campaigns. Bottom row: the sum over all 200, the number on the previous slide.

Estimating a Regression in R

One function: `lm()`

```
1 model <- lm(sales ~ youtube, data = marketing) # "sales as a function of youtube"
2 summary(model)
```

Call:

```
lm(formula = sales ~ youtube, data = marketing)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-10063.2	-2345.4	-229.5	2480.5	8654.8

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	8439.112259	549.411528	15.36	<2e-16 ***
youtube	0.047537	0.002691	17.67	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3910 on 198 degrees of freedom

Multiple R-squared: 0.6119, Adjusted R-squared: 0.6099

F-statistic: 312.1 on 1 and 198 DF, p-value: < 2.2e-16

Find the intercept and the slope in this output. What else is in there?

Reading the output: the estimates

```
1 coef(model)
```

(Intercept)	youtube
8439.11225895	0.04753664

- **Intercept** $\beta_0 = 8,439$: a campaign with **no** YouTube spend is predicted to sell about **8,400 units**
- **Slope** $\beta_1 = 0.0475$: each additional **\$1** on YouTube is associated with **0.0475 more units** sold
- Prediction: with \$100,000 on YouTube, $8,439 + 0.0475 \times 100,000 \approx 13,200$ units

Always translate the slope into **business units**: “one more unit of X goes with β_1 more units of Y ”. If you cannot say it in a sentence a manager understands, you have not read the coefficient yet.

The estimate comes with uncertainty

Our slope comes from **these** 200 campaigns. Another 200 would give a slightly different slope. The **standard error (SE)** measures how much it would move around.

```
1 round(coef(summary(model)), 4)
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	8439.1123	549.4115	15.3603	0
youtube	0.0475	0.0027	17.6676	0

- **Std. Error** of the slope: **0.0027**. The slope is pinned down to roughly 0.0475 ± 0.005
- **t value** = estimate / SE = $0.0475/0.0027 \approx 17.7$: the slope is about 18 standard errors away from zero
- **Pr(>|t|)** is the **p-value**: here so small that it rounds to **0** (two slides back, `summary()` printed it as `<2e-16`)

The question the p-value answers

Hypothesis testing is a way to decide between two claims with data:

- H_0 (the null): **no relationship**, $\beta_1 = 0$. YouTube spend has nothing to do with sales
- H_1 (the alternative): **there is one**, $\beta_1 \neq 0$

The **p-value** answers: *if H_0 were true, how likely is a slope at least this far from zero, just by luck?*

Decision rule

- p-value **< 0.05**: reject H_0 . The coefficient is **statistically significant** (different from zero)
- p-value **≥ 0.05** : fail to reject H_0 . Not enough evidence

R's stars: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, . $p < 0.1$

Smaller SE $\rightarrow$ larger t $\rightarrow$ smaller p $\rightarrow$ stronger evidence against “no relationship”

For YouTube, p is essentially 0: if spend and sales were unrelated, a slope of 0.0475 would almost never show up. So we are confident the relationship is real.

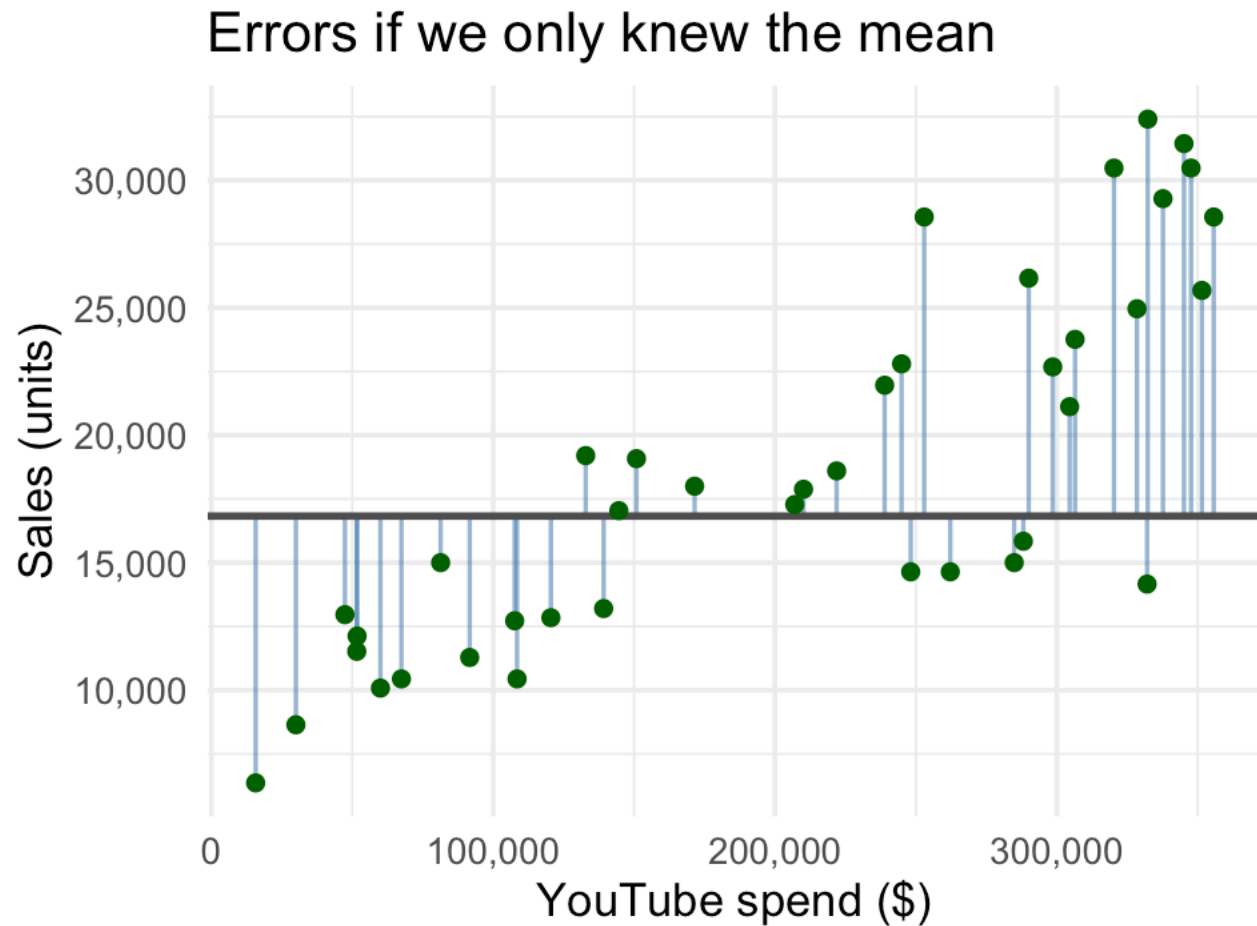
“Significant” is not the same as “big”

Two separate questions about every coefficient:

1. **Is it different from zero?** That is what the p-value answers. It depends on the size of the estimate *and* on how much data you have
2. **Does it matter?** That is about the **size** of the estimate, in business units. The p-value says nothing about it

With 50,000 rows (Airbnb, later today) almost everything is “significant”, including effects too small to care about. With 18 rows almost nothing is, including effects that are real. Read the **size** first, then the stars.

How good is the line? R-squared



Forty of the 200 campaigns. Without X , the best guess for every campaign is the average. How much do the errors shrink once we use the line?

R-squared: reading it

$$R^2 = 1 - \frac{\text{sum of squared errors with the line}}{\text{sum of squared errors with the mean}} = 1 - \frac{3,027,644,040}{7,800,694,200} = 0.61$$

- **Multiple R-squared: 0.61.** YouTube spend explains **61%** of the variation in sales across campaigns. The other 39% is other channels, seasonality, luck
- R^2 runs from 0 (the line is no better than the mean) to 1 (every point on the line)
- **Residual standard error: 3,910.** The typical prediction misses by about **3,900 units**. Same idea as R^2 , but in the units of Y
- **Adjusted R-squared** is R^2 with a small penalty for every predictor you add. Use it to compare models with different numbers of predictors

A low R^2 does not make a coefficient wrong. It says that X alone leaves most of Y unexplained. Whether that matters depends on your goal: **prediction** needs a high R^2 , **understanding one relationship** does not.

The output, decoded

In the output	What it is	How to read it
Estimate	$\hat{\beta}$, the coefficient	“One more unit of X goes with $\hat{\beta}$ more units of Y ”
Std. Error	Precision of the estimate	Smaller is better. Would the estimate move much with new data?
t value	Estimate / SE	How many SEs from zero. Beyond ± 2 is “significant”
Pr(> t)	p-value	Chance of a t this large if the true coefficient is 0
Residual standard error	Typical size of the error	In units of Y
R-squared	Share of the variation in Y explained	0 to 1
F-statistic	Do the predictors explain <i>anything</i> together?	Tiny p-value: yes

Interpreting the Coefficient

What does β_1 tell us?

All else equal (*ceteris paribus*), how much Y changes when X moves by one unit. What “one unit” means depends on how you wrote the model:

Model	Equation	Reading of β_1
Level-level	$Y = \beta_0 + \beta_1 X$	One more unit of $X \rightarrow \beta_1$ more units of Y
Log-level	$\ln Y = \beta_0 + \beta_1 X$	One more unit of $X \rightarrow$ about $100 \times \beta_1$ percent more Y
Level-log	$Y = \beta_0 + \beta_1 \ln X$	1% more $X \rightarrow$ about $\beta_1/100$ more units of Y
Log-log	$\ln Y = \beta_0 + \beta_1 \ln X$	1% more $X \rightarrow$ about β_1 percent more Y (an elasticity)

The percentage readings are approximations that work when β_1 is small (say below 0.2). The exact formula for log-level is $(e^{\beta_1} - 1) \times 100\%$.

Why take logs?

1. Skewed variables. Price, income, sales, number of reviews: many small values, a few huge ones. You saw in week 2 that a log scale makes such distributions readable. The same is true for a regression: without the log, the few huge values pull the line around.

2. Multiplicative relationships. A \$1 price increase means something very different at \$5 and at \$100. In logs, the model works in **percentages**, which is how the relationship usually behaves.

3. Interpretation. Managers think in percentages: “a 10% price increase lowers demand by 8%”. A log-log coefficient **is** that number, the **elasticity**.

Rule of thumb: log a variable that is positive and right-skewed (money, counts). Leave ratings, shares, and dummies as they are.

Logs change the **question** you are asking, not just the fit. Level: “how many more units?”. Log: “what percent more?”. Choose the one your manager needs.

The same relationship, three ways

```
1 m_level <- lm(sales ~ youtube, data = marketing)
2 m_loglev <- lm(log(sales) ~ youtube, data = marketing)
3 m_loglog <- lm(log(sales) ~ log(youtube), data = marketing)
4 stargazer(m_level, m_loglev, m_loglog, type = "text",
5           column.labels = c("level-level", "log-level", "log-log"),
6           omit.stat = c("f", "ser", "adj.rsq"), digits = 4, digits.extra = 4,
7           no.space = TRUE)
```

```
=====
                        Dependent variable:
-----
              sales          log(sales)
             level-level    log-level    log-log
                (1)          (2)          (3)
-----
youtube          0.0475***    0.000003***
                (0.0027)    (0.0000002)
log(youtube)                                0.3550***
                                           (0.0149)
Constant      8,439.1120***    9.0973***    5.4781***
                (549.4115)    (0.0362)    (0.1756)
-----
Observations      200          200          200
R2                0.6119        0.6156        0.7421
=====
Note:              *p<0.1; **p<0.05; ***p<0.01
```

Three ways: reading it

- **Level-level** (column 1): \$1 more on YouTube → **0.0475 more units**
- **Log-level** (column 2): \$1 more on YouTube → about **0.0003% more sales** (100×0.000003)
- **Log-log** (column 3): 1% more YouTube spend → **0.36% more sales**. That is the **elasticity** of sales with respect to YouTube spend

`stargazer()` puts several models side by side, with standard errors in parentheses and stars for significance. This is the table format for Homework 2 and for your project. Column labels, a title, and readable variable names are the difference between a table and a wall of numbers.

Elasticities below 1 mean Y responds less than proportionally: doubling YouTube spend does not double sales.

More than one predictor

$$y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \cdots + \beta_k x_{k,i} + \epsilon_i$$

Everything we said still applies, with one change in the reading: each β_j is the change in Y for one more unit of x_j , **holding the other predictors fixed**.

In R, add predictors to the formula with **+**:

```
1 m_all <- lm(sales ~ youtube + facebook + newspaper, data = marketing)
```

Before you look: last week's charts showed YouTube strong, Facebook noisier, newspaper flat. Which coefficient do you expect to be largest? Which one close to zero?

Three channels at once

```
1 m_all <- lm(sales ~ youtube + facebook + newspaper, data = marketing)
2 stargazer(m_yt, m_all, type = "text", column.labels = c("YouTube only", "all three"),
3           omit.stat = c("f", "ser", "adj.rsq"), digits = 3, no.space = TRUE)
```

=====		
Dependent variable:		

	sales	
	YouTube only	all three
	(1)	(2)

youtube	0.048***	0.046***
	(0.003)	(0.001)
facebook		0.189***
		(0.009)
newspaper		-0.001
		(0.006)
Constant	8,439.112***	3,526.667***
	(549.412)	(374.290)

Observations	200	200
R2	0.612	0.897
=====		
Note:	*p<0.1; **p<0.05; ***p<0.01	

Which channel has the largest coefficient? Which one is essentially zero?

More than one predictor: reading it

- **Facebook** has the largest coefficient: \$1 more on Facebook, holding YouTube and newspaper fixed, goes with about **0.19 more units**. YouTube: 0.046 units per \$1
- **Newspaper**: -0.001 , p-value 0.86. Once YouTube and Facebook are in the model, newspaper spend tells us **nothing** about sales. Remember last week's flat newspaper panel
- R^2 goes from 0.61 to **0.90**: three channels explain 90% of the variation in sales

“Holding the others fixed” is what makes multiple regression useful. Campaigns that spend more on newspaper also spend more on Facebook (correlation 0.35). The single-predictor regression of sales on newspaper would give newspaper credit for Facebook's work.

The Airbnb Data

The dataset

```
1 airbnb <- fread("data/airbnb.csv")
2 names(airbnb)
```

```
[1] "listing_id"      "price"           "bathrooms"       "bedrooms"
[5] "cancellation_policy" "guests_included" "property_type"    "zipcode"
[9] "star_rating"     "reviews_count"   "city"            "room_type"
```

```
1 head(airbnb[, .(price, bedrooms, star_rating, reviews_count, city, room_type)], 4) # a few columns
```

	price	bedrooms	star_rating	reviews_count	city	room_type
	<int>	<int>	<num>	<int>	<char>	<char>
1:	238	2	5.0	79	Los Angeles	Entire home
2:	89	1	5.0	117	Austin	Entire home
3:	89	1	5.0	91	Miami	Private room
4:	175	2	4.5	15	Austin	Entire home

```
1 table(airbnb$city)
```

Austin	Boston	Los Angeles	Miami	New York City
10311	6147	27611	6749	18

50,836 listings with a nightly price, size, star rating, number of reviews, city, and room type. Four real cities and, for reasons lost to history, **18 listings in New York**. Keep that in mind.

Predict before you run

Let's predict **price** as a function of the **number of reviews** a listing has.

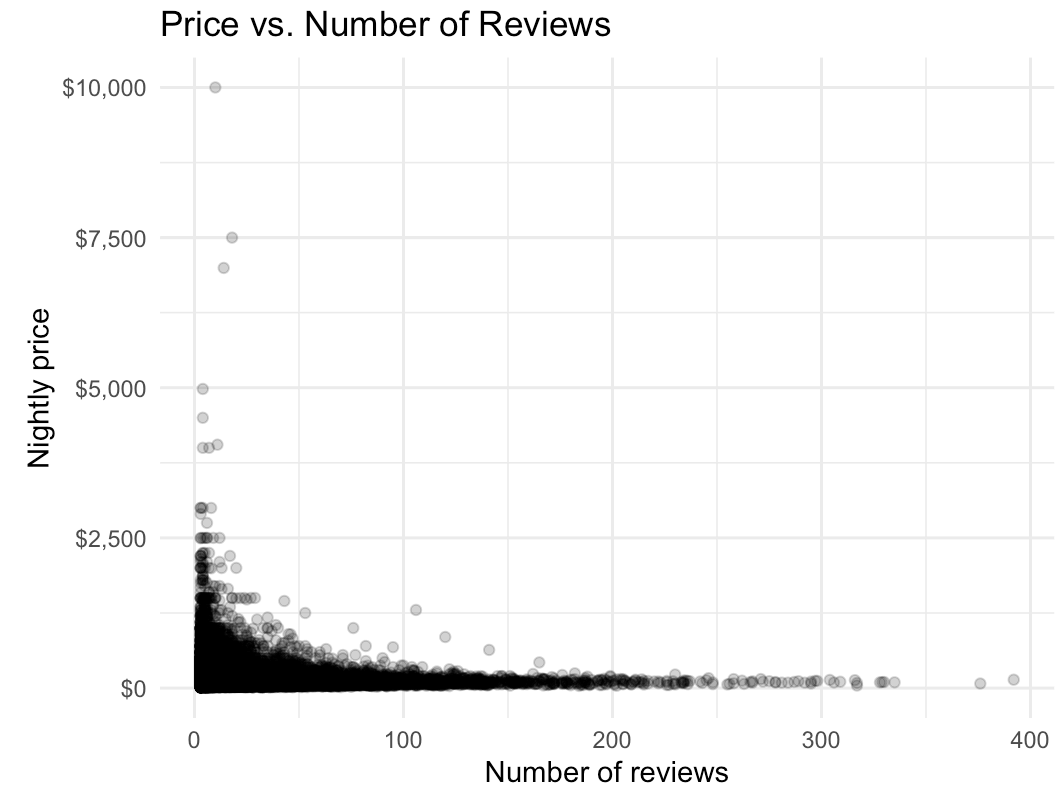
Write down one sentence: **what sign** do you expect for the slope, and **why**?

Two stories, opposite signs:

- More reviews means a popular, established listing, which can charge more (**positive**)
- Cheap places get booked more often, so they collect more reviews (**negative**)

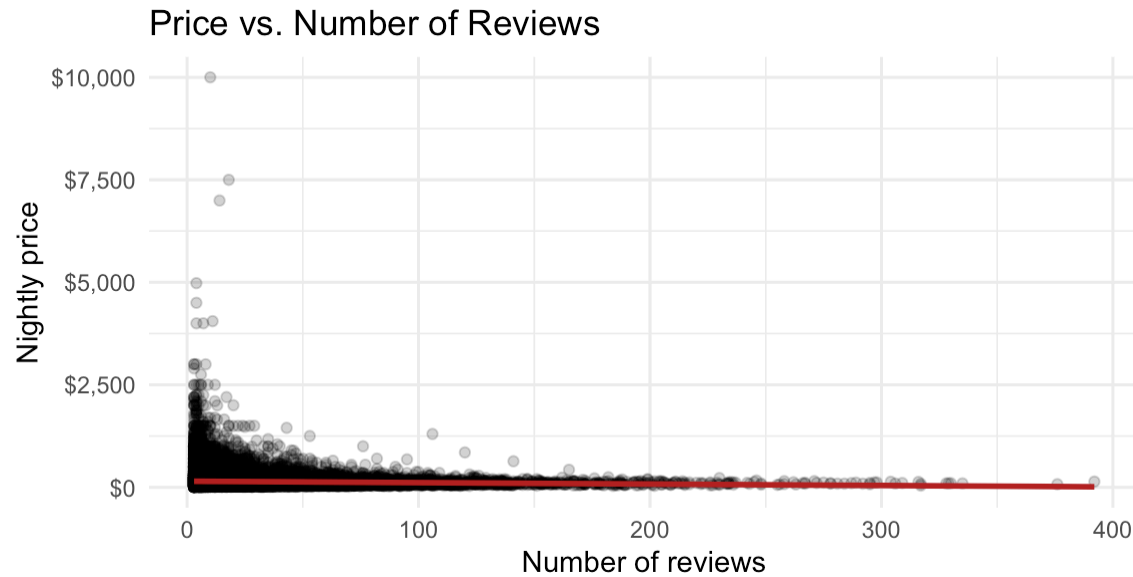
Price vs. number of reviews

```
1 ggplot(airbnb, aes(x = reviews_count, y = price))
2   geom_point(alpha = 0.2) +
3   scale_y_continuous(labels = dollar) +
4   labs(title = "Price vs. Number of Reviews",
5         x = "Number of reviews",
6         y = "Nightly price") +
7   theme_minimal()
```



50,836 points. What do you see? Does a straight line make sense here?

Price vs. reviews: reading it



- Both variables are **heavily skewed**: median price \$100, a few listings at \$10,000; median 8 reviews, a few above 300
- No clear line. If anything, the pricey listings sit at **low** review counts, so the line tilts slightly **down**
- A prime candidate for logs (the exercise on Tuesday, Sept. 22). For now, let's see what the straight line says

The regression

```
1 m1 <- lm(price ~ reviews_count, data = airbnb)
2 summary(m1)
```

Call:

```
lm(formula = price ~ reviews_count, data = airbnb)
```

Residuals:

Min	1Q	Median	3Q	Max
-145.0	-74.9	-39.5	23.3	9855.4

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	148.04066	0.90102	164.30	<2e-16 ***
reviews_count	-0.34672	0.03195	-10.85	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 165.9 on 50834 degrees of freedom

Multiple R-squared: 0.002311, Adjusted R-squared: 0.002291

F-statistic: 117.8 on 1 and 50834 DF, p-value: < 2.2e-16

Read the slope, the p-value, and the R-squared. Do they agree with the scatter plot?

The regression: reading it

- **Intercept**: a listing with zero reviews is predicted to cost **\$148**
- **Slope** -0.35 : each additional review is associated with a price **35 cents lower**. Ten more reviews: \$3.50 lower. That is tiny next to a median price of \$100
- **p-value** $< 2e-16$, three stars: statistically significant. With 50,000 listings, even a 35-cent slope is measured precisely
- **R-squared** **0.002**: reviews explain **0.2%** of the variation in price
- **Residual standard error** **\$166**: the typical miss is larger than the typical price

Significant, and useless for prediction. The stars say the slope is not zero. The size and the R^2 say it does not matter. Both readings are correct, and you need both.

A table you can put in a report

```
1 stargazer(m1, type = "text",
2           title = "Regression of Price on Number of Reviews",
3           dep.var.labels = "Nightly price (dollars)",
4           covariate.labels = "Number of reviews",
5           omit.stat = c("f", "ser", "adj.rsq"), digits = 2, no.space = TRUE)
```

Regression of Price on Number of Reviews

=====

Dependent variable:

Nightly price (dollars)

Number of reviews	-0.35*** (0.03)
Constant	148.04*** (0.90)

Observations	50,836
R2	0.002

=====

Note: *p<0.1; **p<0.05; ***p<0.01

Same numbers as `summary()`, laid out for a reader. Standard errors in parentheses, stars for significance, N and R^2 at the bottom.

Categorical Predictors

Text in a regression?

City is a word, not a number. R handles this with a **factor**: a categorical variable stored as numbers with labels attached.

```
1 city <- factor(c("Miami", "Austin", "Miami", "Boston"))  
2 city
```

```
[1] Miami Austin Miami Boston  
Levels: Austin Boston Miami
```

```
1 as.integer(city)
```

```
[1] 3 1 3 2
```

- R stores 1, 2, 3 and shows you the labels. Levels are sorted alphabetically unless you say otherwise
- `lm()` converts a text column into a factor automatically. But **what does a slope on “city” mean?** Let’s look

Price by city

```
1 m_city <- lm(price ~ city, data = airbnb)
2 stargazer(m_city, type = "text", title = "Regression of Price on City",
3           dep.var.labels = "Nightly price (dollars)", omit.stat = c("f", "ser", "adj.rsq"),
4           digits = 2, no.space = TRUE)
```

Regression of Price on City

```
=====
                        Dependent variable:
                        -----
                        Nightly price (dollars)
                        -----
cityBoston              -17.60***
                        (2.67)
cityLos Angeles         -36.77***
                        (1.91)
cityMiami               -40.93***
                        (2.59)
cityNew York City       -28.17
                        (39.03)
Constant                169.95***
                        (1.63)
-----
Observations            50,836
R2                      0.01
=====
```

Five cities, four coefficients. Where did Austin go? And what is the constant?

Price by city: reading it

Behind the scenes, R turns city into **dummy variables** (0/1), one per city, and **drops one**: the **base level**, Austin (first alphabetically).

Listing	City	Boston	LA	Miami	NYC
1	Austin	0	0	0	0
2	Boston	1	0	0	0
3	LA	0	1	0	0
4	Miami	0	0	1	0
5	NYC	0	0	0	1

- The **constant** (\$169.95) is the average price in the base city, **Austin**
- Each coefficient is the **difference from Austin**: Boston \$17.60 cheaper, Los Angeles \$36.77 cheaper, Miami \$40.93 cheaper. All three precisely estimated (three stars)
- **New York**: \$28 cheaper, p-value 0.47, **not** significant. Not because New York is like Austin, but because there are **18** listings: the standard error is \$39, versus \$2–3 for the other cities

“Not significant” can mean “no difference” or “not enough data”. Look at the standard error and the group size before you conclude anything.

Choosing the base level

```
1 airbnb$city <- relevel(factor(airbnb$city), ref = "Los Angeles")
2 m_city2 <- lm(price ~ city, data = airbnb)
3 stargazer(m_city, m_city2, type = "text", column.labels = c("base: Austin", "base: Los Angeles"),
4           dep.var.labels = "Nightly price (dollars)", omit.stat = c("f", "ser", "adj.rsq"),
5           digits = 2, no.space = TRUE)
```

Dependent variable:		
Nightly price (dollars)		
	base: Austin (1)	base: Los Angeles (2)
cityAustin		36.77*** (1.91)
cityBoston	-17.60*** (2.67)	19.18*** (2.33)
cityLos Angeles	-36.77*** (1.91)	
cityMiami	-40.93*** (2.59)	-4.16* (2.25)
cityNew York City	-28.17 (39.03)	8.61 (39.01)
Constant	169.95*** (1.63)	133.17*** (1.00)

Same data, same fit (same R^2), different comparison. Pick the base your reader cares about: the largest group, or the “status quo” option.

Regression Is Still Not Causation

What the coefficient does and does not say

Price is \$37 lower in Los Angeles than in Austin. Reviews go with slightly lower prices. Facebook spend goes with more sales. All of these are **associations**. None of them tells you what happens if you *change* X .

- A regression holds fixed **only the predictors you put in**. Everything you left out (listing quality, neighborhood, season) is still free to move with X and Y at the same time
- Adding predictors helps (newspaper's "effect" vanished once Facebook was in). It does not settle the question. Weeks 11–12 are about designs that do

Words matter. Write “a one-unit increase in X **is associated with** a β_1 change in Y ”, not “ X **causes** Y ”, unless you have a reason to believe it, and can say why.

Wrap-up

What's next

- **Tuesday, Sept. 22: logit**, regression for yes/no outcomes (will a listing get booked? will a customer churn?). Then the **Airbnb exercise** on your laptop: logs, more predictors, city and room type, and where you would buy a property. Bring the week 1 setup working
- **Homework 2** (due Thursday, Oct. 8): the same tools on a box-office dataset, OLS and logit, with **stargazer** tables and an executive memo

Before Tuesday, run today's script on the Airbnb data and try one thing on your own: replace **reviews_count** with **bedrooms** in **m1**. Predict the slope first, then explain back what the intercept means.

Wrap-up

Code and data to reproduce today's regressions, on the course website:

- [w4-code.zip](#): one-click download of everything below
- [w4-1-ols-class.R](#): every chart and regression shown today
- [data/airbnb.csv](#): the Airbnb dataset (also used in the Airbnb exercise on Tuesday, Sept. 22)

Required reading: Chapter [3.4](#) of *R for Marketing Students*. Optional: [Lecture 6 of Data Storytelling for Marketers](#) and Chapter 7 of *R for Marketing Research and Analytics*.

Questions?